

Module 2: Querying Data - SELECT Fundamentals

Introduction to Data Retrieval

The `SELECT` statement is the most fundamental and frequently used SQL command. It allows you to retrieve data from one or more tables in a database. Mastering `SELECT` is essential because most of your work with databases will involve querying and analyzing data.

Sample Database Setup

Throughout this module, we'll use a **Students** table from a School database. Here's the structure:

```
-- Create the database
CREATE DATABASE School;
GO

-- Use the database
USE School;
GO

-- Create the Students table
CREATE TABLE Students
(
    Roll INT,
    StudentName VARCHAR(50),
    English DECIMAL(5, 2),
    Maths DECIMAL(5, 2),
    Science DECIMAL(5, 2),
    Standard INT,
    Division VARCHAR(1),
    Address VARCHAR(50),
    DateOfBirth DATE
);
```

Output:

Database 'School' created successfully.

Table 'Students' created successfully.

 Note:

In SQL Server, `GO` is a batch separator that tells SQL Server to execute all statements above it before moving to the next batch. It's not a SQL statement itself, but a command recognized by SQL Server utilities.

The SELECT Statement

The basic SELECT statement retrieves data from a table. The simplest form retrieves all columns and all rows.

Basic Syntax

```
SELECT column1, column2, ...
FROM table_name;
```

Selecting All Columns

The asterisk (*) is a wildcard that represents "all columns" in the table.

```
SELECT * FROM Students;
```

Output:

Roll	StudentName	English	Maths	Science	Standard	Division	Address	DateOfBirth
1	Amit Kumar	85.50	92.00	88.75	10	A	Mumbai	2009-05-15

Roll	StudentName	English	Maths	Science	Standard	Division	Address	DateOfBirth
2	Priya Sharma	78.00	85.50	90.25	10	A	Delhi	2009-08-22
3	Rahul Verma	92.00	88.00	85.50	10	B	Pune	2009-03-10
4	Sneha Patel	65.00	70.50	68.00	9	A	Mumbai	2010-11-30
5	Arjun Singh	88.50	95.00	92.75	10	A	Pune	2009-07-18

Selecting Specific Columns

Instead of retrieving all columns, you can specify exactly which columns you need. This is more efficient and provides clearer, more focused results.

```
SELECT StudentName, English, Maths, Science
FROM Students;
```

Output:

StudentName	English	Maths	Science
Amit Kumar	85.50	92.00	88.75
Priya Sharma	78.00	85.50	90.25
Rahul Verma	92.00	88.00	85.50
Sneha Patel	65.00	70.50	68.00

StudentName	English	Maths	Science
Arjun Singh	88.50	95.00	92.75

 Tip:

Always select only the columns you need rather than using `SELECT *`. This improves query performance, reduces network traffic, and makes your code more maintainable.

Sorting Results with ORDER BY

The `ORDER BY` clause sorts the result set based on one or more columns. By default, sorting is in ascending order (A to Z, 0 to 9).

Ascending Order (Default)

```
SELECT * FROM Students
ORDER BY StudentName;
```

Output:

Roll	StudentName	English	Maths	Science
1	Amit Kumar	85.50	92.00	88.75
5	Arjun Singh	88.50	95.00	92.75
2	Priya Sharma	78.00	85.50	90.25
3	Rahul Verma	92.00	88.00	85.50
4	Sneha Patel	65.00	70.50	68.00

Students sorted alphabetically by name

Descending Order

Use the `DESC` keyword to sort in descending order (Z to A, 9 to 0).

```
SELECT * FROM Students
ORDER BY StudentName DESC;
```

Output:

Roll	StudentName	English	Maths
4	Sneha Patel	65.00	70.50
3	Rahul Verma	92.00	88.00
2	Priya Sharma	78.00	85.50
5	Arjun Singh	88.50	95.00
1	Amit Kumar	85.50	92.00

Students sorted in reverse alphabetical order

Sorting by Multiple Columns

You can sort by multiple columns. The second column is used as a tiebreaker when the first column has duplicate values.

```
SELECT * FROM Students
ORDER BY Standard DESC, Maths DESC;
```

Output:

Roll	StudentName	Standard	Maths
5	Arjun Singh	10	95.00
1	Amit Kumar	10	92.00
3	Rahul Verma	10	88.00
2	Priya Sharma	10	85.50
4	Sneha Patel	9	70.50

First sorted by Standard (highest first), then by Maths marks (highest first within each standard)

Limiting Results

When working with large datasets, you often need to retrieve only a specific number of rows. SQL Server provides two methods for this: **TOP** and **OFFSET/FETCH**.

Using TOP

The **TOP** clause returns the first N rows from the result set.

```
-- Get the first 10 students
SELECT TOP 10 * FROM Students
ORDER BY Roll;
```

Output:

(10 rows returned)

Roll	StudentName	Maths
1	Amit Kumar	92.00

Roll	StudentName	Maths
2	Priya Sharma	85.50
3	Rahul Verma	88.00
... (7 more rows)		

Finding Top Performers

```
-- Find the 5 highest scorers in Maths
SELECT TOP 5 StudentName, Maths, Standard
FROM Students
ORDER BY Maths DESC;
```

Output:

StudentName	Maths	Standard
Arjun Singh	95.00	10
Amit Kumar	92.00	10
Rahul Verma	88.00	10
Priya Sharma	85.50	10
Sneha Patel	70.50	9

Note:

When using **TOP** with **ORDER BY**, the sorting happens first, then TOP selects from the sorted results. This is important for getting meaningful "top N" results.

Using OFFSET and FETCH (Pagination)

The `OFFSET` and `FETCH` clauses allow you to skip rows and retrieve a specific range. This is particularly useful for pagination in applications.

Syntax

```
SELECT columns
FROM table_name
ORDER BY column_name
OFFSET n ROWS
FETCH NEXT m ROWS ONLY;
```

Example: Get rows 6 to 20

```
SELECT Roll, StudentName, Maths
FROM Students
ORDER BY Roll
OFFSET 5 ROWS           -- Skip the first 5 rows
FETCH NEXT 15 ROWS ONLY; -- Get the next 15 rows
```

Output:

Roll	StudentName	Maths
6	Kavya Reddy	82.00
7	Ravi Kumar	76.50
8	Ananya Iyer	89.00
... (12 more rows)		
20	Sanjay Nair	79.50

Returns rows 6 through 20 (15 rows total)

 Important:

`OFFSET/FETCH` requires an `ORDER BY` clause. Without it, you'll get a syntax error. This makes sense because "skip 5 rows" is only meaningful if the rows have a defined order.

Pagination Example

Here's how you would implement page-based navigation (e.g., 10 students per page):

```
-- Page 1 (rows 1-10)
SELECT * FROM Students
ORDER BY Roll
OFFSET 0 ROWS
FETCH NEXT 10 ROWS ONLY;

-- Page 2 (rows 11-20)
SELECT * FROM Students
ORDER BY Roll
OFFSET 10 ROWS
FETCH NEXT 10 ROWS ONLY;

-- Page 3 (rows 21-30)
SELECT * FROM Students
ORDER BY Roll
OFFSET 20 ROWS
FETCH NEXT 10 ROWS ONLY;
```

 Tip:

Formula for pagination: `OFFSET (PageNumber - 1) * PageSize ROWS FETCH NEXT PageSize ROWS ONLY`

Example: For page 5 with 10 items per page: `OFFSET (5-1)*10 = OFFSET 40 ROWS FETCH NEXT 10 ROWS ONLY`

Filtering Data with WHERE

The `WHERE` clause filters rows based on specified conditions. Only rows that satisfy the condition(s) are included in the result set.

Basic WHERE Syntax

```
SELECT columns
FROM table_name
WHERE condition;
```

Comparison Operators

Operator	Description	Example
<code>=</code>	Equal to	<code>Standard = 10</code>
<code><></code> or <code>!=</code>	Not equal to	<code>Standard <> 7</code>
<code>></code>	Greater than	<code>Maths > 80</code>
<code><</code>	Less than	<code>Science < 35</code>
<code>>=</code>	Greater than or equal to	<code>English >= 35</code>
<code><=</code>	Less than or equal to	<code>Maths <= 50</code>

Example 1: Students Who Passed in Maths

```
-- Passing marks is 35
SELECT StudentName, Maths
FROM Students
WHERE Maths >= 35;
```

Output:

StudentName	Maths
Amit Kumar	92.00
Priya Sharma	85.50
Rahul Verma	88.00
Sneha Patel	70.50
Arjun Singh	95.00

Example 2: Students from a Specific Standard

```
SELECT * FROM Students
WHERE Standard = 6;
```

Output:

(All students from Standard 6 displayed)

Example 3: Students from a Specific City

```
SELECT StudentName, Address, Standard
FROM Students
WHERE Address = 'Pune';
```

Output:

StudentName	Address	Standard
Rahul Verma	Pune	10

StudentName	Address	Standard
Arjun Singh	Pune	10

 Note:

String comparisons in SQL Server are **case-insensitive by default** (depending on collation). This means 'Pune', 'pune', and 'PUNE' are treated as equal.

Example 4: Students Who Failed

```
-- Students who scored less than 35 in Science
SELECT StudentName, Science
FROM Students
WHERE Science < 35;
```

Output:

StudentName	Science
Deepak Joshi	32.50
Meena Gupta	28.00

Logical Operators

Logical operators allow you to combine multiple conditions in a WHERE clause.

AND Operator

Returns rows where **all** conditions are true.

```
-- Students who passed in all three subjects
SELECT StudentName, English, Maths, Science
FROM Students
WHERE English >= 35
      AND Maths >= 35
      AND Science >= 35;
```

Output:

StudentName	English	Maths	Science
Amit Kumar	85.50	92.00	88.75
Priya Sharma	78.00	85.50	90.25
Rahul Verma	92.00	88.00	85.50
Sneha Patel	65.00	70.50	68.00
Arjun Singh	88.50	95.00	92.75

OR Operator

Returns rows where **at least one** condition is true.

```
-- Students from Standard 5 or Standard 9
SELECT StudentName, Standard
FROM Students
WHERE Standard = 5
      OR Standard = 9;
```

Output:

StudentName	Standard
Sneha Patel	9

StudentName	Standard
Vikram Rao	5
Aditi Shah	9
Karan Mehta	5

NOT Operator

Negates a condition (returns rows where the condition is false).

```
-- All students except those in Standard 7
SELECT StudentName, Standard
FROM Students
WHERE NOT Standard = 7;

-- Alternative syntax (preferred)
SELECT StudentName, Standard
FROM Students
WHERE Standard <> 7;
```

Output:

(All students from standards other than 7 are displayed)

Combining Multiple Conditions

```
-- Students from Pune who failed in Maths
SELECT StudentName, Address, Maths
FROM Students
WHERE Maths < 35
      AND Address = 'Pune';
```

Output:

StudentName	Address	Maths
Rohan Desai	Pune	32.00

```
-- Students from Standard 6, Division C
SELECT StudentName, Standard, Division
FROM Students
WHERE Standard = 6
      AND Division = 'C';
```

Output:

StudentName	Standard	Division
Neha Kulkarni	6	C
Tushar Jain	6	C

BETWEEN Operator

The **BETWEEN** operator selects values within a given range (inclusive of both boundaries).

Syntax

```
WHERE column_name BETWEEN value1 AND value2;
```

Example: Range Query

```
-- Students who scored between 35 and 50 in Science, from Pune
SELECT StudentName, Science, Address
FROM Students
WHERE Science BETWEEN 35 AND 50
      AND Address = 'Pune';
```

Output:

StudentName	Science	Address
Pooja Malik	42.50	Pune
Saurabh Patil	38.00	Pune
Divya Rane	47.75	Pune

Note:

`BETWEEN 35 AND 50` is equivalent to `Science >= 35 AND Science <= 50`. The `BETWEEN` operator is more concise and readable for range queries.

IN Operator

The `IN` operator allows you to specify multiple values in a `WHERE` clause. It's a shorthand for multiple `OR` conditions.

Syntax

```
WHERE column_name IN (value1, value2, value3, ...);
```


Example: Multiple Values

```
-- Students from Standards 3, 6, or 8 who are from Pune
SELECT StudentName, Standard, Address, Maths
FROM Students
WHERE Standard IN (3, 6, 8)
      AND Address = 'Pune'
ORDER BY Maths;
```

Output:

StudentName	Standard	Address	Maths
Riya Agarwal	6	Pune	65.00
Nikhil Bhat	8	Pune	72.50
Aarav Deshmukh	3	Pune	78.00

The above query is much cleaner than:

```
WHERE (Standard = 3 OR Standard = 6 OR Standard = 8)
      AND Address = 'Pune'
```

Using Parentheses for Clarity

When mixing AND and OR operators, use parentheses to make the logic clear:

```
-- Top 5 students with lowest Maths scores from Standards 6, 3, or 8 in Pune
SELECT TOP 5 StudentName, Standard, Maths, Address
FROM Students
WHERE (Standard = 6 OR Standard = 3 OR Standard = 8)
      AND Address = 'Pune'
ORDER BY Maths ASC;
```

 Tip:

Without parentheses, AND takes precedence over OR, which can lead to unexpected results. Always use parentheses to make your intention explicit.

Pattern Matching with LIKE

The `LIKE` operator is used for pattern matching with text data. It uses wildcards to define patterns.

Wildcards

Wildcard	Description	Example
<code>%</code>	Matches zero or more characters	<code>'A%'</code> matches "Amit", "Arjun", "A"
<code>_</code>	Matches exactly one character	<code>'A_it'</code> matches "Amit", but not "Arjit"
<code>[]</code>	Matches any single character within brackets	<code>'[AP]%'</code> matches names starting with A or P
<code>[^]</code>	Matches any single character NOT in brackets	<code>'[^AP]%'</code> matches names NOT starting with A or P

Example 1: Names Starting with 'A'

```
SELECT StudentName
FROM Students
WHERE StudentName LIKE 'A%';
```

Output:

StudentName

Amit Kumar

Arjun Singh

Ananya Iyer

Aditi Shah

Aarav Deshmukh

Example 2: Names with 'R' as Third Letter

```
-- Names that start with A and have R as the third letter
SELECT StudentName
FROM Students
WHERE StudentName LIKE 'A_r%';
```

Output:

StudentName

Arjun Singh

Example 3: Names Ending with Specific Letter

```
-- Names that end with 'a'
SELECT StudentName
FROM Students
WHERE StudentName LIKE '%a';
```

Output:

StudentName
Priya Sharma
Ananya Iyer
Kavya Reddy

Example 4: Names Containing Specific Pattern

```
-- Students with 'oo' in their names
SELECT StudentName
FROM Students
WHERE StudentName LIKE '%oo%';
```

Output:

StudentName
Roopa Das

Example 5: Names with Specific Length

```
-- Names with exactly 5 characters
SELECT StudentName
FROM Students
WHERE StudentName LIKE '_____';
```

Output:

StudentName
Sneha

StudentName
Karan

Example 6: Using Character Sets

```
-- Names starting with A, P, or R
SELECT StudentName
FROM Students
WHERE StudentName LIKE '[APR]%';
```

Output:

StudentName
Amit Kumar
Priya Sharma
Rahul Verma
Arjun Singh

Query Execution Order

Understanding the order in which SQL Server processes query clauses helps you write better queries and debug issues.

Logical Processing Order

1. **FROM** - Identify the table(s)
2. **WHERE** - Filter rows based on conditions
3. **SELECT** - Choose columns to display

4. **ORDER BY** - Sort the result set
5. **OFFSET/FETCH** or **TOP** - Limit the number of rows returned

 Note:

This is the **logical** order. SQL Server's query optimizer may execute operations in a different physical order for performance, but the result is always as if they were processed in this logical order.

Example Demonstrating Order

```
SELECT TOP 5 StudentName, Maths -- 4. Select these columns, limit to 5
FROM Students                  -- 1. From this table
WHERE Address = 'Pune'         -- 2. Filter for Pune students
ORDER BY Maths DESC;           -- 3. Sort by Maths descending
```

Best Practices

- **Be specific with columns:** Use `SELECT column1, column2` instead of `SELECT *`
- **Use meaningful ORDER BY:** Always order by a relevant column for consistent, predictable results
- **OFFSET/FETCH needs ORDER BY:** Required for pagination to work correctly
- **Use BETWEEN for ranges:** More readable than combining `>=` and `<=`
- **Use IN for multiple values:** Cleaner than multiple OR conditions
- **Use parentheses:** Make complex conditions explicit and clear
- **LIKE is case-insensitive by default:** In standard SQL Server collations
- **Index WHERE columns:** Frequently filtered columns should be indexed for performance

Common Mistakes to Avoid

- **Forgetting ORDER BY with OFFSET/FETCH:** Results in a syntax error
- **Mixing AND/OR without parentheses:** Can produce unexpected results
- **Using = NULL instead of IS NULL:** NULL comparisons require IS NULL, not =
- **Case-sensitive string comparisons:** Be aware of your database's collation settings

- **LIKE without wildcards:** `LIKE 'value'` is the same as `= 'value'` ; use wildcards for pattern matching

Summary

In this module, we covered the fundamental techniques for querying data:

- Using `SELECT` to retrieve all or specific columns
- Sorting results with `ORDER BY` (ascending and descending)
- Limiting results with `TOP` and `OFFSET/FETCH`
- Filtering data with `WHERE` and comparison operators
- Combining conditions with `AND` , `OR` , and `NOT`
- Using `BETWEEN` for range queries
- Using `IN` for multiple value matching
- Pattern matching with `LIKE` and wildcards
- Understanding query execution order

These skills form the foundation of data retrieval in SQL. In the next module, we'll learn how to modify existing data and enforce data integrity using constraints.

End of Module 2: Querying Data - SELECT Fundamentals